Project 2 Proposal: Echo Command

ECE 1895 – Junior Design Fundamentals

Tengyue Zhang Xiaodong Deng Chuxuan Zhou

1 Project overview

Echo Command is a version of the Bop It reaction game, built as two units. The game has a tabletop command-and-score station that generates audio commands for the player to respond to using a wireless handheld controller. The station judges and acts according to the player's response. The handheld controller has one control for each command, namely, "Push it!", "Flip it!" or "Twist it!" During a game, if the handheld controller response is correct and the response time is within the time limit (2.5 seconds at the start), a green LED is lit to indicate a successful response, and one point is added to the score on the OLED display. In case of wrong or delayed response, the red LED is lit indicating GAME OVER with the final score. Each time the player gets one point, the time limit is shortened (down to 0.7 seconds), therefore the game gets more difficult at every successful attempt, up to a win at 99 points.

The team recognized the value of Tengyue Zhang's two-unit concept and improved on it together. Both units are powered by their own rechargeable battery with their own power switch, and a new game is offered only when the station can detect the controller. In addition, the player uses a START button on the handheld controller to confirm the new game. Also, the station does not have the preliminary proposal's single RGB indicator but two 10 mm LEDs, one red and one green, which makes it easier to read the result from a distance. One more improvement is in the enclosure design; both units are planned together with their electronics so that the final product looks and feels like a toy, not an electronic piece of equipment. The two units communicate over ESP-NOW, which allows the handheld controller to be held comfortably by the player, and the same player to observe the score.

2 System block diagram

Figure 1 shows the major elements. Each unit is built around an Adafruit ESP32 Feather V2 board that combines the ESP32 radio, a USB-C LiPo charger and a 3.3 V regulator onto a single board [1]; using the same board for both units ensures that we do not have to maintain multiple toolchains or pin maps. The controller reads four inputs and transmits each detected action to the station, while the latter unit is in control of the game, voice prompts, display and LEDs.

3 Subsystem design

3.1 Handheld controller

The controller has three command inputs plus a START button (Table 1). Each button and the toggle ground a GPIO pin and use the ESP32's built-in pull-up, so no resistors are needed. The potentiometer is placed at A2 (GPIO34), which is an ADC1 pin, because the ESP32 cannot read its ADC2 pins while running the Wi-Fi radio [1]. Push It and START are arcade buttons containing LEDs fed from GPIO26 and GPIO25, which indicate what the station expects, e.g., START is lit when the station is waiting to start. When using 3.3 V GPIO pins to power the LEDs, they will be dimmer as they expect 5 V (a transistor stage from the battery is the fallback). The controller does not judge any answers; it simply generates an ACTION packet when it detects a Push, Flip, or Twist, a START packet when START is pressed, and a heartbeat packet every 200 ms, so timing and scoring stay in one place (the station).

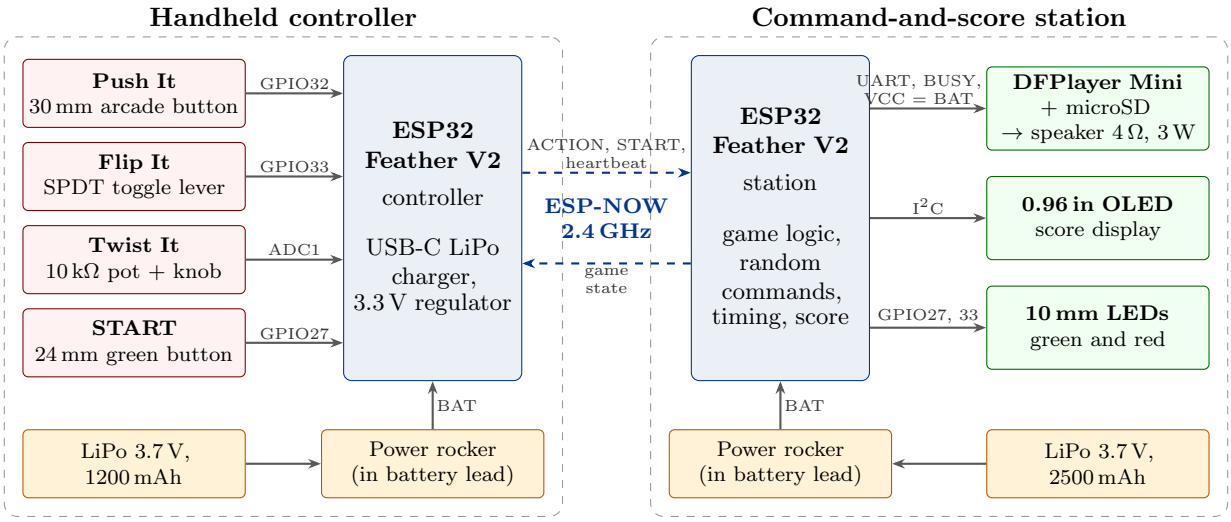


Figure 1: System block diagram. Solid arrows are wires, dashed arrows the ESP-NOW link. The DFPlayer runs from the switched battery (Feather BAT pin), not the 3.3 V regulator.

Table 1: Controller inputs and how each one is detected.

Command	Part	Pin	Detection
Push It	30 mm arcade button, red LED	GPIO32	Debounced press (falling edge, 20 ms).
Flip It	SPDT panel toggle lever	GPIO33	Debounced change of lever position, either way.
Twist It	10 kΩ linear panel pot	A2 (GPIO34)	Average of 8 readings moves over ∼800 counts (20 % of full scale) from a stored reference, which then resets.
START	24 mm arcade button, green LED	GPIO27	Debounced press, used only when the station asks.

3.2 Command-and-score station

The sound prompts originate from a DFPlayer Mini connected with a microSD card, while ESP32 controls this device via `Serial1` at 9600 baud rate: TX pin (GPIO8) is connected to DFPlayer's RX pin through a 1 kΩ series resistor as per DFRobot's suggestions [2], whereas RX pin is GPIO7. The DFPlayer's BUSY output is connected to GPIO32. BUSY pin stays LOW during the play of voice clip, thus the station always knows for sure when the voice clip has ended, and it can start its response timer, exactly as the preliminary proposal defined [**?**]. DFPlayer's amplifier drives a 4 Ω, 3 W enclosed speaker directly. Also, the team is going to record such clips as: "Push it!", "Flip it!", "Twist it!", "Press START to play", a short chime indicating the correct answer, "Game over", and "You win!" and store them in the `/mp3` folder as `0001.mp3` through `0007.mp3`. The score and game-over screen are displayed on a 0.96-inch 128×64 OLED screen which connects via cable to a STEMMA QT (I^2C) connector in Feather, hence there is no need for soldering. The 10 mm green and red LEDs are connected to GPIO27 and GPIO33 respectively, with about 100 Ω in series, which provides about 10 mA for each LED.

3.3 Wireless link

Both Feathers are paired using MAC addresses, over the ESP-NOW protocol, which does not need a router and sends short low-latency packets. Each packet carries a type and action code, also contains battery level of controller and sequence number (Table 2). The station ignores all packets

with repeated sequence numbers, thus packets that the radio resends are never counted more than once. The station sees the controller as a connected one only if it receives a heartbeat in the last second. If there is no heartbeat signal, it will not ask for a new game and in case of heartbeats stopping during the game, the game ends, displaying "controller lost" instead of being scored as a miss.

Table 2: ESP-NOW packet types.

Type	Direction	Sent when	Contents
HEARTBEAT	controller → station	every 200 ms	battery level, sequence number
START	controller → station	START pressed	sequence number
ACTION	controller → station	Push, Flip or Twist	action code, sequence number
STATE	station → controller	state changes	waiting, ready, playing, game over

3.4 Power

Each device features a single-cell 3.7 V Lithium Polymer battery with a capacity of 1200 mAh used in the controller and 2500 mAh in the station, each battery plugged into the Feather board via its JST-PH battery connector. The power switch is connected in a splice to an extension cable with JST-PH connectors and runs between the battery and the Feather board; thus, OFF on the switch actually disconnects the battery itself, meaning that the switch needs to be ON while Feather charges the battery through the USB-C connector. The DFPlayer in the station is powered from that switched battery (3.5–4.2 V, still in its 3.2–5.0 V range [2]), and not from the Feather board's 3.3 V regulator, which supplies about 500 mA peak and provides half of that for the ESP32 device [1]. Both the Feather boards are able to read the voltage of their batteries using the GPIO35 voltage divider circuit, and the station gives low battery warnings for either of the units. The radio equipment uses most of the power, hence we are expecting around 8–10 hours of play at one charge for the controller and more than 10 hours for the station; we will be measuring the actual current using the bench supply.

4 Software plan and pseudo-code

The state machine is run by the station as shown in Figure 2; Listing 1 shows the `main()` loop of this state machine, and Listing 2 shows the controller's `main()` loop. Each major task is a function contained in a separately testable module (i.e., `radio`, `inputs`, `game`, `audio`, `display`, `power`, `leds`); Table 3 shows who leads each module. The response window is $T = \max(0.7\,\text{s},\ 2.5\,\text{s} - 25\,\text{ms} \times \mathit{score})$, where T reaches its minimum of 0.7 s at the score of 72; the step will be adjusted by playing the game. An action that comes in before the command clip has finished playing is immediately judged, and the timer begins only after the clip has completed.

5 Enclosure design

Both enclosures will be 3D-printed in Pitt blue and gold with two-piece PLA shells with 2.5 mm walls. Panels have openings only for controls, display, LEDs, speaker grille, power rockers, and USB-C charging ports. The sizes of the openings are determined from component datasheets. For example, arcade buttons require 30 mm and 24 mm holes, the rockers require 19.4×13.1 mm cut-outs, a toggle mount requires a 1/4-inch opening, a potentiometer requires a 7 mm opening, and LEDs require 10 mm openings. The 3D-printed Twist It knob is pressed on the knurled 6 mm shaft of the potentiometer. The batteries are inserted into padded pockets that avoid the screws on either half of the enclosing shells. The controller (Figure 3) has rounded grips, Push It at the right thumb, the Flip It lever under the left thumb, Twist It on the top edge, and START placed in the center.

Listing 1: Station `main()` (pseudo-code).

```
void setup() {
  initRadio(); initAudio(); initDisplay(); initLeds();   // ESP-NOW, DFPlayer, OLED, LEDs
  randomSeed(esp_random());
  state = WAIT_LINK;
}

void loop() {
  switch (state) {
    case WAIT_LINK:                                     // controller off or out of range
      showWaiting();
      if (controllerOnline()) { promptStart(); state = ASK_START; }
      break;
    case ASK_START:                                     // OLED + "Press START to play"
      if (!controllerOnline())     state = WAIT_LINK;
      else if (takeStartRequest()) { newGame(); state = ISSUE_CMD; }   // score 0, T 2.5 s
      break;
    case ISSUE_CMD:
      if (clipPlaying()) break;                         // let the last chime finish
      cmd = pickCommand();                              // PUSH, FLIP or TWIST
      clearActions();                                   // drop actions sent earlier
      playClip(cmd);
      state = WAIT_ACTION;
      break;
    case WAIT_ACTION:
      if (!controllerOnline()) { endGame(LINK_LOST); break; }
      if (clipJustEnded()) startTimer();                // T counts from the end of the clip
      a = takeAction();                                 // newest unread action, or NONE
      if (a != NONE)                      judge(a == cmd);
      else if (timerElapsed() > windowMs) judge(false);                // too late
      break;
  }
  if (lowBattery()) showBatteryWarning();
}

void judge(bool correct) {
  if (!correct) { endGame(MISS); return; }
  score++;  flashLed(GREEN);  playClip(CLIP_CORRECT);  showScore(score);
  windowMs = max(700, 2500 - 25 * score);               // shorter window after every point
  if (score == 99) endGame(WIN); else state = ISSUE_CMD;
}

void endGame(Reason r) {                                // MISS, WIN or LINK_LOST
  setLed(r == WIN ? GREEN : RED);  playClip(r == WIN ? CLIP_WIN : CLIP_GAME_OVER);
  showGameOver(score, r);  sendState(GAME_OVER);        // final score stays on the OLED
  state = (r == LINK_LOST) ? WAIT_LINK : ASK_START;
}
```

Listing 2: Controller `main()` (pseudo-code).

```
void setup() {
  initInputs();  initRadio();  initButtonLeds();   // pull-ups, ADC on A2, station as peer
  lastFlip = readToggle();
  twistRef = readPotFiltered();
}
void loop() {
  if (millis() - lastBeat >= 200) {                 // heartbeat every 200 ms
    sendPacket(HEARTBEAT, batteryLevel());  lastBeat = millis();
  }
  if (startPressed())   sendPacket(START);
  if (pushPressed())    sendPacket(ACTION, PUSH);   // debounced press
  if (flipChanged())    sendPacket(ACTION, FLIP);   // lever moved either way
  if (twistDetected()) sendPacket(ACTION, TWIST);   // knob turned far enough
  updateButtonLeds(stationState);                   // e.g. START lit while the station asks
}
bool twistDetected() {
  int v = readPotFiltered();                        // average of 8 ADC readings
  if (abs(v - twistRef) > TWIST_THRESHOLD) { twistRef = v; return true; }
  return false;
}
```

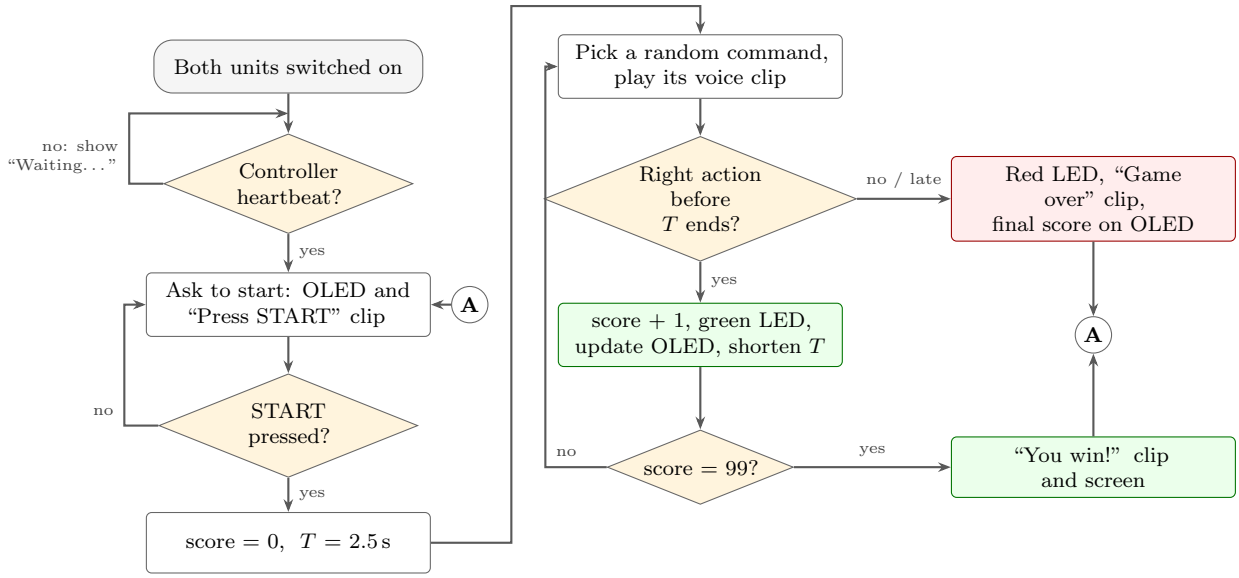
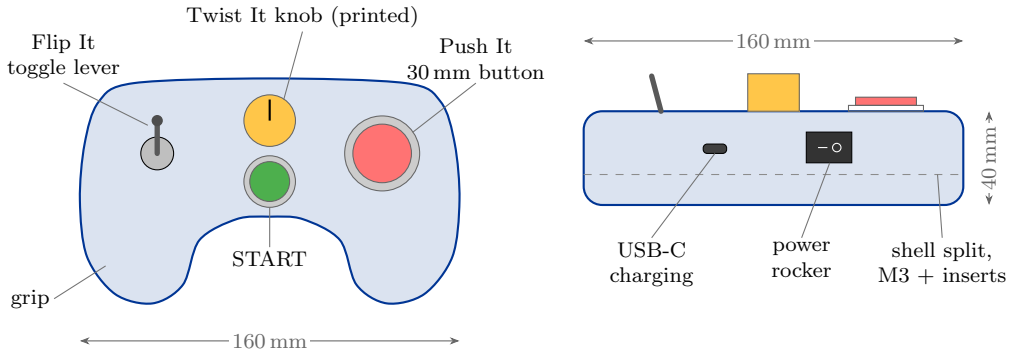


Figure 2: Station game flow. The two circles marked A are the same point: after a game ends, the station goes back to asking for START. If the controller's heartbeat stops at any point, the station returns to "Waiting for controller".

The station (Figure 4) is shaped as a wedge to face the player, with the two LEDs and display on the sloped face above the speaker grille. The enclosure dimensions will vary according to the measurements of the boards after their different components have been soldered. Both enclosure halves are joined with M3 screws fastened to heat-set inserts to allow repairs.



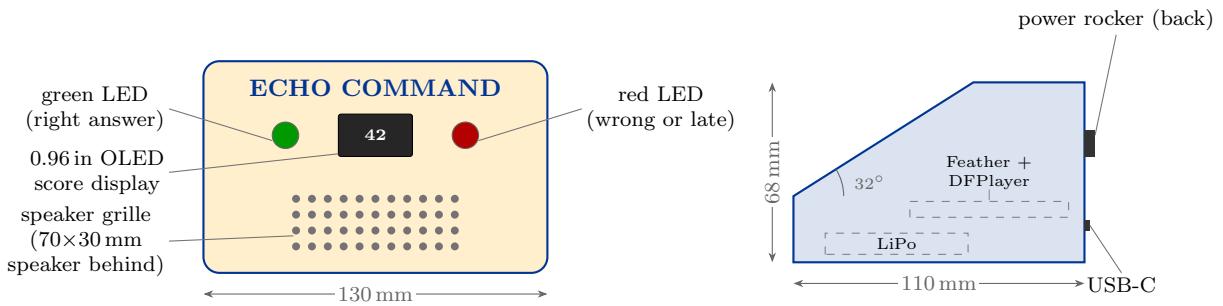
(a) Top view **(b) Front edge (faces away from player)**

Figure 3: Preliminary sketch of the handheld controller enclosure, about $160 \times 95 \times 40$ mm (not final dimensions).

6 Team roles

Once the DigiKey order has been received, the team will build the units. Each of the people on the team will help build the single pair of units, with the team sharing the testing and reporting. Each also has their area of responsibility (see Table 3) in which they will lead and provide firmware modules. The firmware does not need to be developed together for the two halves, since the units only need to agree on the packet format (Table 2).

Assuming that the radio link, voice prompts and input thresholds all pass breadboard testing, the initial breadboard designs will be upgraded to soldered perfboard for both units. Once the soldered



(a) Sloped front face **(b) Side profile**

Figure 4: Preliminary sketch of the command-and-score station enclosure (not to final dimensions). The sloped face points the display and LEDs at the player.

boards are measured, everything can go together in an enclosure printed around them for testing before finalizing the design.

Table 3: Team roles and firmware ownership.

Member	Leads	Main tasks	Firmware modules
Tengyue Zhang	Handheld controller	Input wiring and detection (debounce, twist threshold), ESP-NOW packets and link check, controller firmware, fitting the controls into the shell.	`radio`, `inputs`
Xiaodong Deng	Command-and-score station	Game state machine and timing, DFPlayer audio and recording the voice clips, OLED screens, station firmware.	`game`, `audio`, `display`
Chuxuan Zhou	Power, enclosures, integration	Batteries, charging and power switching, LED drive, enclosure design and 3D printing, soldered boards, parts ordering and BOM, full-system tests.	`power`, `leds`

7 Preliminary bill of materials

Table 4: Preliminary DigiKey order (USD).

#	Qty	DigiKey PN	Part	Used in	Each	Total
1	2	1528-5400-ND	ESP32 Feather V2 (Adafruit 5400)	both	19.95	39.90
2	1	1528-1838-ND	LiPo 3.7 V, 1200 mAh (Adafruit 258)	controller	9.95	9.95
3	1	1528-1840-ND	LiPo 3.7 V, 2500 mAh (Adafruit 328)	station	14.95	14.95
4	2	1528-1494-ND	JST-PH extension cable (Adafruit 1131)	both	1.95	3.90
5	2	EG5617-ND	Snap-in rocker, SPST (E-Switch RA11131121)	both	0.64	1.28
6	1	1528-2316-ND	30 mm arcade button, red LED (Adafruit 3489)	controller	2.50	2.50
7	1	EG2350-ND	SPDT toggle (E-Switch 100SP1T1B1M1QEH)	controller	3.08	3.08
8	1	PDB181-E415K-103B-ND	10 kΩ linear panel pot (Bourns PDB181)	controller	1.63	1.63
9	1	1528-2513-ND	24 mm arcade button, green LED (Adafruit 3433)	controller	2.50	2.50
10	1	1738-1041-ND	DFPlayer Mini (DFRobot DFR0299)	station	5.90	5.90
11	1	3351-ND	Enclosed speaker, 4 Ω 3 W (Adafruit 3351)	station	3.95	3.95
12	1	1528-1220-ND	0.96 in OLED, STEMMA QT (Adafruit 326)	station	17.50	17.50
13	1	1528-4210-ND	STEMMA QT cable, 100 mm (Adafruit 4210)	station	0.95	0.95
14	2	754-1898-ND	10 mm green LED (Kingbright WP813GD)	station	0.33	0.66
15	2	WP813ID-ND	10 mm red LED (Kingbright WP813ID)	station	0.41	0.82
			Total			**109.47**

References

[1] Adafruit Industries, "Adafruit ESP32 Feather V2: Pinouts," https://learn.adafruit.com/adafruit-esp32-feather-v2/pinouts.

[2] DFRobot, "DFPlayer Mini SKU:DFR0299," https://wiki.dfrobot.com/DFPlayer_Mini_SKU_DFR0299.